

give me detailed step to build application like spiiny for selling second hand cars

Absolutely. If by “like **Spinny**” you mean an application for **buying and selling used cars**, with car listings, inspection, pricing, customer leads, booking, payments, documents, and an admin system, you should think of it as **three applications + backend**, not just one mobile app.

A good target architecture is:

Customer App/Web → Backend/API → Admin Panel → Inspection/Operations App → Database + Storage + Payments + Notifications

Below is a practical step-by-step roadmap for building it.

1. First decide your business model

Before writing code, decide which model you are building.

Model A — Marketplace

Owners/dealers list their cars.

```
Seller
  ↓
List Car
  ↓
Customer searches
  ↓
Customer contacts seller
  ↓
Deal happens between them
```

This is the easiest MVP.

Model B — Inventory-based

Your company buys cars, inspects them, refurbishes them and sells them.

```
graph TD; A[Car Owner] --> B[Sell Car]; B --> C[Inspection]; C --> D[Purchase car]; D --> E[Refurbishment]; E --> F[Your Inventory]; F --> G[Customer]; G --> H[Booking / Purchase]; H --> I[Delivery];
```

This is much closer to the Spinny-style experience.

Model C — Hybrid

Allow:

- Your own inventory
- Dealer inventory
- Individual sellers

I would recommend **Model C eventually**, but start with **Model B or A for MVP**.

2. Applications you need

Don't build everything in one application.

Customer application

For:

- Browse cars
- Search
- Filters
- Car details
- Compare
- Favorites
- Book test drive

- Contact sales
- Buy/reserve
- Financing
- Insurance
- Delivery tracking

Could be:

Flutter / React Native

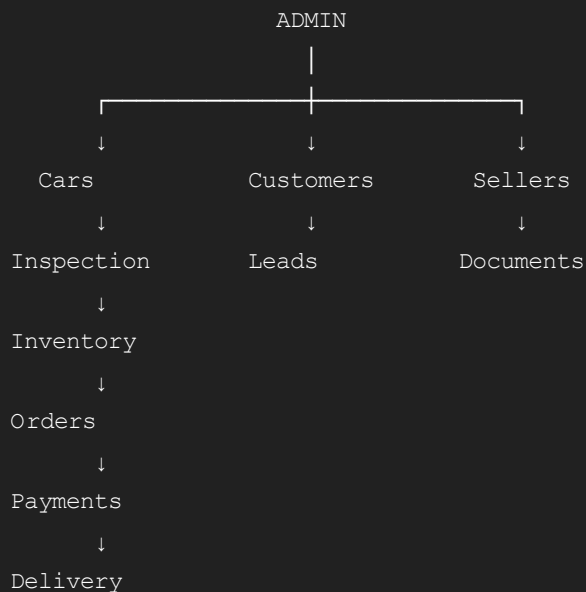
or initially:

React/Next.js web application

3. Admin dashboard

This is extremely important.

Your admin dashboard controls the entire business.



Admin modules:

Dashboard

Show:

- Total cars
- Available cars
- Sold cars
- New leads
- Test drives

- Bookings
- Revenue
- Pending inspections
- Cars awaiting approval

Example:

Used Car Admin

Cars

Leads

Bookings

Revenue

342

128

27

₹42.6 L

Recent Activity

Toyota Innova

Inspection Pending

Honda City

Test Drive Booked

Hyundai Creta

Payment Pending

Maruti Swift

Sold

4. Inspection application

If you want a serious used-car platform, this is one of the biggest differentiators.

An inspector should be able to use a phone/tablet.

For example:

```
Inspection

Vehicle
↓
Exterior
↓
Interior
↓
Engine
↓
Transmission
↓
Electrical
↓
Tyres
↓
```

```
Suspension
↓
AC
↓
Accident history
↓
Documents
↓
OBD Scan
↓
Final Score
```

Inspector can upload:

- Photos
- Videos
- Inspection measurements
- Damage locations
- Tyre condition
- Engine condition
- Battery condition
- Odometer reading
- RC details
- Insurance details

5. Recommended technology stack

Since you already have experience with **Python and GUI development**, Python is a very good choice for your backend.

I'd recommend:

Frontend

Web

```
Next.js
React
TypeScript
Tailwind CSS
```

Mobile

```
Flutter
```

or React Native.

If you're starting alone, I'd actually recommend:

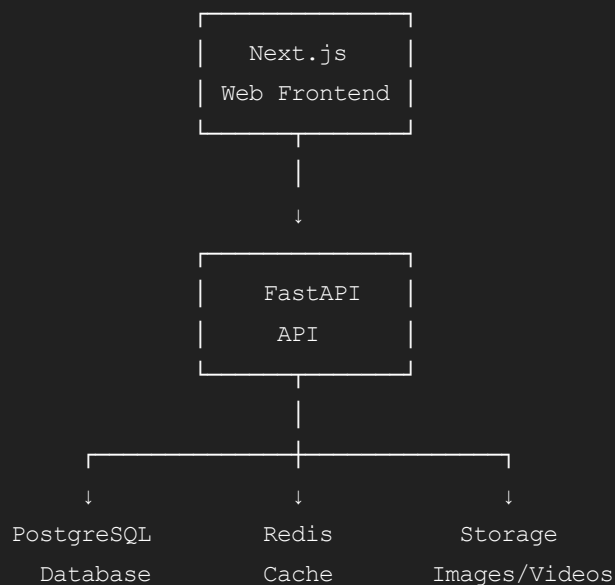
Next.js first → Flutter later.

6. Backend

Use:

```
Python
FastAPI
PostgreSQL
Redis
Celery
```

Architecture:



7. Create the project structure

For example:

```
used_car_platform/
|
├─ backend/
|   └─ app/
|       └─ main.py
|       |
|       └─ api/
```

```
| | | └─ auth.py
| | | └─ cars.py
| | | └─ users.py
| | | └─ sellers.py
| | | └─ inspections.py
| | | └─ bookings.py
| | | └─ payments.py
| | | └─ orders.py
| | |
| | | └─ models/
| | | └─ schemas/
| | | └─ services/
| | | └─ database/
| | | └─ utils/
| | └─ requirements.txt
|
└─ frontend/
  └─ app/
  └─ components/
  └─ pages/
  └─ services/
└─ mobile/
└─ admin/
```

8. Design your database first

This is extremely important.

Don't start by designing screens.

Start with your database.

9. Users table

```
users

id
name
phone
email
password_hash
role
```

created at

Roles:

CUSTOMER
SELLER
INSPECTOR
ADMIN
SALES
MANAGER

10. Cars table

```
cars

id
make
model
variant
year
registration_year
fuel_type
transmission
body_type
kms_driven
owner_count
color
engine_cc
registration_city
registration_number
price
status
created_at
updated_at
```

Example:

Make: Hyundai
Model: Creta
Variant: SX(O)
Year: 2022
Fuel: Diesel
Transmission: Automatic
KM: 38,500
Owners: 1
Price: ₹14,50,000

11. Car images

Don't put images directly into PostgreSQL.

Use object storage.

For example:

```
car_images

id
car_id
image_url
image_type
sort_order
created_at
```

Types:

```
FRONT
REAR
LEFT
RIGHT
INTERIOR
DASHBOARD
ENGINE
TYRE
DAMAGE
DOCUMENT
```

12. Car features

You need a separate system for features.

```
car_features

id
car_id
feature_name
feature_value
```

Example:

```
Sunroof          Yes
Cruise Control  Yes
ABS              Yes
```

Airbags	6
Apple CarPlay	Yes
Android Auto	Yes
360 Camera	Yes

13. Inspection database

This can become one of your most important modules.

```
inspections

id
car_id
inspector_id
inspection_date
overall_score
engine_score
exterior_score
interior_score
tyre_score
electrical_score
suspension_score
transmission_score
status
remarks
```

Then individual inspection items:

```
inspection_items

id
inspection_id
category
item
status
severity
remarks
```

Example:

```
Exterior
├─ Front bumper      PASS
├─ Bonnet             MINOR_DAMAGE
├─ Left door          PASS
└─ Rear bumper        REPAIR_REQUIRED
```

14. Inspection scoring

You can create your own scoring system.

For example:

Exterior	15%
Interior	10%
Engine	20%
Transmission	15%
Suspension	10%
Electrical	10%
Tyres	5%
AC	5%
Documents	10%

Then:

```
Overall Score =  
Exterior × 0.15 +  
Interior × 0.10 +  
Engine × 0.20 +  
...
```

Display:

8.7 / 10	
Engine	9.2
Exterior	8.5
Interior	8.9
Tyres	8.0
Electrical	9.0
Suspension	8.4

This creates a strong **trust factor** for customers.

15. Vehicle listing workflow

A car should not immediately become public.

Use states.

```
LEAD  
↓  
SELLER CONTACTED
```

↓
INSPECTION SCHEDULED
↓
INSPECTION COMPLETED
↓
PURCHASE APPROVED
↓
ACQUIRED
↓
REFURBISHMENT
↓
READY FOR SALE
↓
LIVE
↓
RESERVED
↓
SOLD

Your database should have:

car_status

with values like:

DRAFT
INSPECTION_PENDING
INSPECTION_COMPLETED
REFURBISHMENT
READY
AVAILABLE
RESERVED
SOLD
ARCHIVED

16. Customer home page

Your home page should be simple.

Find your perfect used car

[Search Make / Model]

Location

[Bangalore ▼]

Budget

[₹5L] - [₹15L]

Popular searches

SUV Sedan Hatchback
Automatic Diesel Petrol

Recommended Cars

[Car Image]

2022 Hyundai Creta
SX(O) Automatic

38,500 km | Diesel | 1 Owner

₹14.50 Lakh

[View Car]

17. Search system

Users should be able to search by:

Basic

- Make
- Model
- Price
- Location

Advanced

- Year
- Fuel
- Transmission
- KM
- Owners
- Body type
- Color

Features

- Sunroof
- 360 camera

- Cruise control
 - Apple CarPlay
 - Android Auto
 - Ventilated seats
-

18. Build filtering correctly

Example API:

```
http
```

```
GET /api/cars
```

Parameters:

```
make=Hyundai  
model=Creta  
min_price=800000  
max_price=1500000  
fuel=diesel  
transmission=automatic  
min_year=2021  
max_km=50000  
city=Bangalore
```

Backend generates a PostgreSQL query.

Don't fetch every car and filter them in Python.

Use database indexing.

For example:

```
INDEX (price)  
INDEX (make)  
INDEX (model)  
INDEX (city)  
INDEX (year)  
INDEX (fuel_type)  
INDEX (transmission)
```

19. Car details page

This is one of the most important screens.

Layout:

[Large Car Image]

● ● ● ● ● ● ●

2022 Hyundai Creta SX(O)

₹14,50,000

38,500 km

Diesel

Automatic

1st Owner

Bangalore

[Book Test Drive]

[Reserve Car]

Why buy this car?

- ✓ 1 Owner
- ✓ Full inspection
- ✓ No major accident
- ✓ Service history available
- ✓ RC verified

Inspection Score

8.7 / 10

Engine		9.2
Exterior		8.5
Interior		8.9

Features

- ✓ Sunroof
- ✓ 360 Camera
- ✓ Cruise Control
- ✓ Android Auto

20. Favorites

Create:

```
favorites

id
user_id
car_id
created_at
```

API:

```
POST /cars/{id}/favorite
DELETE /cars/{id}/favorite
GET /users/me/favorites
```

21. Test-drive booking

Create:

```
test_drive_bookings

id
user_id
car_id
date
time
location
status
assigned_salesperson
created_at
```

Workflow:

```
Customer
  ↓
Select car
  ↓
Book test drive
  ↓
Select date/time
  ↓
Salesperson assigned
  ↓
Confirmation
```


↓
Test drive
↓
Feedback

22. Lead management / CRM

Every important customer action should generate a lead.

Examples:

Viewed car
↓
Clicked Contact
↓
Lead created

Lead table:

```
leads

id
customer_id
car_id
source
status
assigned_to
priority
created_at
updated_at
```

Statuses:

```
NEW
CONTACTED
INTERESTED
TEST_DRIVE
NEGOTIATION
PAYMENT
WON
LOST
```

Admin can see:

Customer	Car	Status

Rahul	Creta	Test Drive

Arun	City	Contacted
Kiran	Verna	Negotiation

23. Sell-your-car feature

This is important if you want customers to sell cars.

Page:

```
Sell Your Car

Registration Number

[ KA01AB1234 ]

[ Continue ]
```

Then:

```
Car Details
↓
KM
↓
Owner Count
↓
Condition
↓
Photos
↓
Location
↓
Expected Price
↓
Inspection
```

You can later build an automatic valuation engine.

24. Used-car valuation engine

This is where your application can become much more advanced.

Input:

```
Make
Model
```

```
Variant
Year
KM
Fuel
Transmission
City
Owners
Condition
Service history
Accident history
```

Output:

```
Estimated market value

₹8.4L - ₹9.1L

Recommended buying price

₹8.65L

Expected selling price

₹9.25L
```

Start with a rules-based system.

Later use ML.

25. ML pricing model

Once you have enough historical data:

```
X = [
    age,
    kms,
    brand,
    model,
    variant,
    fuel,
    transmission,
    city,
    owners,
    accident_history,
    service_history,
    inspection_score
]
```

Target:

```
selling_price
```

Start with:

```
Random Forest  
XGBoost  
LightGBM
```

You can compare models using:

```
MAE  
RMSE  
MAPE  
R2
```

Don't start with ML on day one.

First collect reliable data.

26. Payment system

When customer wants to reserve a vehicle:

```
Car  
↓  
Reserve  
↓  
Payment  
↓  
Reservation created  
↓  
Car status = RESERVED
```

You need:

```
payments  
  
id  
user_id  
car_id  
order_id  
amount  
payment_gateway  
transaction_id
```

For India, integrate a suitable payment gateway such as Razorpay or another provider appropriate to your business and compliance requirements.

27. Financing

Eventually:

Car Price
₹12,00,000

Down Payment
₹2,00,000

Loan
₹10,00,000

Tenure
60 months

Estimated EMI
₹ _____

Then:

Apply for Finance

You can integrate financing partners later.

28. Insurance

After purchase:

Insurance

Current policy

Expiry

Insurer

[Renew Insurance]

This can become an additional revenue stream.

29. Documents

For each car:

```
RC
Insurance
PUC
Service Records
Inspection Report
Invoice
Warranty
```

Store documents securely in object storage.

Don't expose raw document storage URLs publicly.

30. Notifications

Implement:

SMS

For:

- OTP
- Test drive confirmation
- Payment confirmation
- Delivery

Email

For:

- Booking
- Invoice
- Inspection report

Push notifications

For:

```
Price changed
New similar car
Booking reminder
Payment reminder
Delivery update
```

31. Login system

Start with mobile OTP.

```
Enter mobile
  ↓
OTP
  ↓
Verify
  ↓
Create/Login user
```

Backend should issue:

```
Access Token
Refresh Token
```

Don't store passwords in plain text if you later offer password login.

32. Admin permissions

Don't make every admin user a super-admin.

Example:

```
Super Admin
|
├─ Inventory Manager
├─ Sales Manager
├─ Inspection Manager
├─ Finance
└─ Operations
```

Permissions:

```
cars:create
cars:edit
cars:delete

inspection:create
inspection:approve

orders:view
orders:approve

payments:view
```

33. Image management

Used-car applications will have **a lot of images**.

Don't save:

```
image binary → PostgreSQL
```

Instead:

```
Mobile
  ↓
Upload
  ↓
Object Storage
  ↓
URL
  ↓
Database
```

You can use services such as:

- AWS S3
- Cloudflare R2
- Google Cloud Storage

Also generate thumbnails.

```
original.jpg
medium.jpg
thumbnail.jpg
```

This dramatically improves page performance.

34. Security

You need to implement:

Backend

```
JWT
RBAC
Rate limiting
Input validation
SQL injection protection
```


CORS
HTTPS
Audit logs

Payment

Never store:

Card number
CVV
UPI credentials

Let the payment provider handle sensitive payment information.

Admin

Use:

2FA
Strong passwords
Session expiration
Login monitoring

35. API architecture

Your backend could eventually look like:

```
/api/v1

/auth
  /send-otp
  /verify-otp

/users
  /me

/cars
  GET
  POST
  /{id}
  /{id}/images
  /{id}/features

/search
  /cars

/favorites
  GET
```

```
POST
DELETE
```

```
/inspections
  POST
  GET
  /{id}
```

```
/test-drives
  POST
  GET
  /{id}
```

```
/leads
  GET
  POST
  /{id}
```

```
/orders
  POST
  GET
  /{id}
```

```
/payments
  POST
  /verify
```

```
/sell-car
  POST
```

```
/notifications
```

36. Admin API

Separate admin endpoints logically:

```
/admin/dashboard
```

```
/admin/cars
/admin/cars/{id}
```

```
/admin/inspections
/admin/inspections/{id}
```

```
/admin/leads
/admin/orders
/admin/payments
```

37. Recommended development sequence

Do not attempt to build everything simultaneously.

Build in this order.

Phase 1 — Foundation

Step 1

Create Git repository.

```
used-car-platform
```

Step 2

Set up:

```
Python  
FastAPI  
PostgreSQL  
SQLAlchemy  
Alembic
```

Step 3

Create:

```
users  
cars  
car_images  
car_features
```

Step 4

Build authentication.

Step 5

Build CRUD APIs.

38. Phase 2 — Customer website

Build:

```
Home
↓
Search
↓
Listing
↓
Car Details
↓
Login
↓
Favorite
↓
Test Drive
```

At this point you already have a usable MVP.

39. Phase 3 — Admin panel

Build:

```
Dashboard
Cars
Add Car
Edit Car
Images
Customers
Leads
Bookings
```

This lets you actually operate the platform.

40. Phase 4 — Inspection

Build:

```
Inspector Login
↓
Assigned Cars
↓
Inspection
↓
Photos
↓
```

Checklist

↓

Score

↓

Submit

Admin:

Inspection Pending

↓

Review

↓

Approve

↓

Publish Car

41. Phase 5 — Sell Your Car

Build:

Registration Number

↓

Vehicle information

↓

Owner details

↓

Condition

↓

Expected price

↓

Upload photos

↓

Schedule inspection

Then admin receives:

New Sell-Car Lead

42. Phase 6 — Transactions

Implement:

Reserve Car

↓

Payment

↓
Order
↓
Documentation
↓
Delivery
↓
Sold

43. Phase 7 — Advanced features

Then add:

AI

- Car damage detection
- Number plate recognition
- OCR of documents
- Automated inspection reports
- Price prediction
- Recommendation engine

For example:

Customer searches:

Hyundai Creta under ₹15 lakh

Recommendation engine can return:

Because you viewed:

Creta SX(O)

You may also like:

Kia Seltos

Hyundai Alcazar

Kia Sonet

44. AI vehicle inspection

This is particularly interesting for your application.

Customer/inspector uploads:

```
Front
Rear
Left
Right
Interior
Tyres
Engine
```

Computer vision could identify:

```
Scratch
Dent
Broken light
Paint damage
Crack
Rust
Tyre wear
```

Then:

```
AI Detection
    ↓
Human Inspector Verification
    ↓
Final Inspection Report
```

Don't make AI the final authority for safety/condition decisions. Use it as an assistant to the human inspection process.

45. Recommendation engine

Store user activity:

```
user_id
car_id
event
timestamp
```

Events:

```
VIEW
SEARCH
FAVORITE
CONTACT
```

TEST_DRIVE
RESERVE

Then initially use simple rules:

Viewed SUV
+
Budget ₹10-15L
+
Automatic
+
Bangalore

→ Recommend similar SUVs

Later move to ML recommendations.

46. Analytics

Your admin dashboard should eventually show:

Traffic
↓
Car Views
↓
Leads
↓
Test Drives
↓
Reservations
↓
Purchases

Calculate:

View → Lead conversion
Lead → Test Drive conversion
Test Drive → Purchase conversion

Example:

10,000 car views

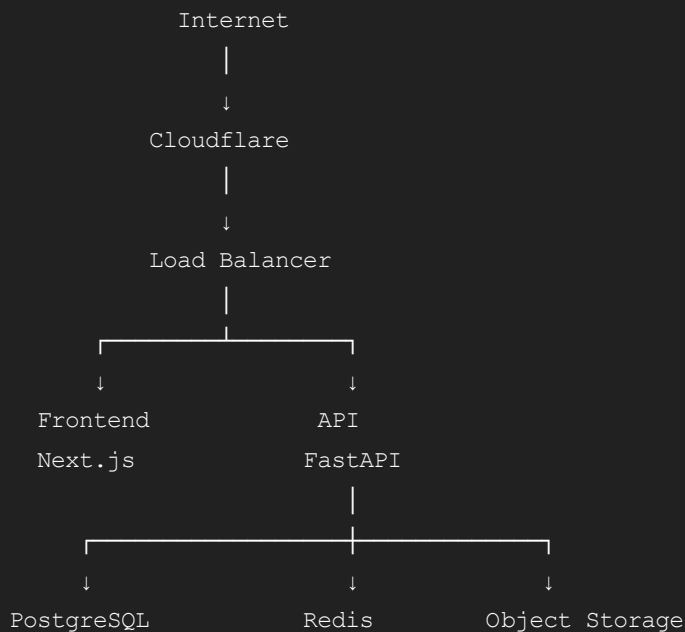
1,200 leads
↓
450 test drives

↓
110 purchases

This tells you where your business is losing customers.

47. Deployment architecture

When you're ready:



Use Docker:

```
docker-compose.yml
```

Services:

```
frontend
backend
postgres
redis
worker
```

48. Development environment

Since you're already familiar with Python, I'd use:

```
VS Code
Python 3.12+
FastAPI
```

PostgreSQL
Git
Docker
Node.js
Next.js

For API testing:

Postman

For database:

pgAdmin

49. Suggested MVP technology stack

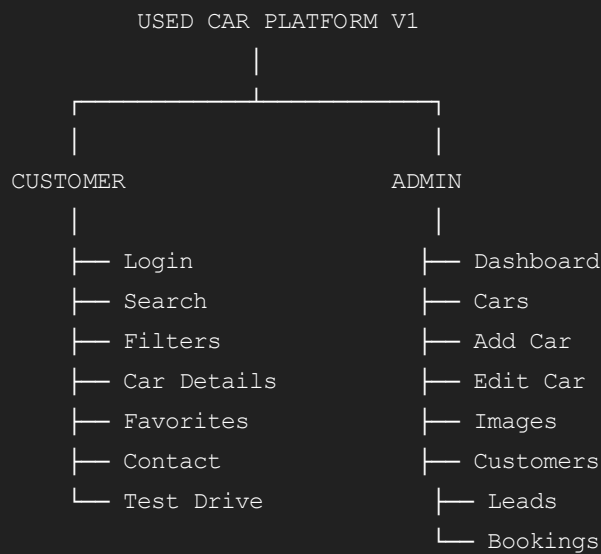
If you're building this yourself, **don't over-engineer it**.

Component	Technology
Website	Next.js
Mobile later	Flutter
Backend	FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Authentication	OTP + JWT
Cache	Redis
Background jobs	Celery
Images	S3/R2
Admin	Next.js
Payments	Payment gateway
Maps	Google Maps/Mapbox
Notifications	Firebase

Component	Technology
Deployment	Docker + Cloud
Analytics	GA4/custom analytics

50. Your first version should NOT contain everything

I'd build **Version 1** like this:



That's enough to launch a basic platform.

51. Then Version 2

V2

```

Sell Your Car
+
Inspection App
+
Inspection Report
+
Price Estimation
+
Payments
+
Orders
  
```

52. Then Version 3

V3

AI Inspection

+

AI Pricing

+

Recommendations

+

Finance

+

Insurance

+

Delivery Tracking

+

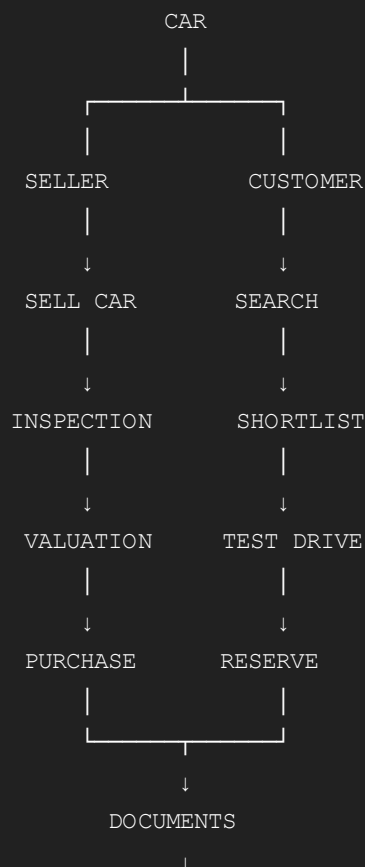
Warranty

53. Most important part: business workflow

If you're trying to build something **actually comparable to Spinny**, don't think of it as:

"I need to make a used-car website."

Think of it as:



PAYMENT
↓
DELIVERY
↓
SOLD

The software should be built around this workflow.

What I would recommend for you specifically

Given that you're already comfortable with **Python** and **GUI/tool development**, don't start by trying to build a huge production-grade Spinny clone.

Start with:

FastAPI + PostgreSQL + Next.js

and build this exact sequence:

```
1. Database
   ↓
2. FastAPI backend
   ↓
3. Admin login
   ↓
4. Add/Edit car
   ↓
5. Upload car photos
   ↓
6. Customer car listing
   ↓
7. Search + filters
   ↓
8. Car details
   ↓
9. Customer login
   ↓
10. Favorites
   ↓
11. Test-drive booking
   ↓
12. Admin lead management
```

Once those 12 pieces work, you have a **real working used-car marketplace**, rather than just a UI prototype.

If you want to actually start developing it, I can take you through the next level **step-by-step from an empty VS Code folder**, including the **PostgreSQL database design**, **complete FastAPI project structure**, **API endpoints**, **Next.js UI**, **admin dashboard**, and **code for each module**, in the order you should build them.

For building a second-hand car selling app, here's one web deployment option.



 Cloudflare

Ad

Deploy web applications for free
Build full-stack apps with your framework of choice, including React, Vue, Svelte, Next &...

Ads do not influence the answers you get from ChatGPT. Your chats stay private.
[Learn about ads and personalization](#)

ChatGPT can make mistakes. Check important info.